RBQ10 비전 RL 구현 총정리 GAST Ray+IR 교사 직결 구조 · 2026-10-18

목적 → 방법 → 입출력 → 지형처리기 → Actor-Critic·PPO 학습 → 평가 도구(E1·RL Replay·MuJoCo) → 배포 순서로 정리한 전달용 문서. 기준 모델은 **Ray+IR 3906**(sha f3e7afe5)이고, 2026-10-18 기준 `gd_lab_vrl` main(8c61d0d)과 `gd_rbq10_deploy_vrl` main(f0109e0)의 코드·설정 파일에서 확인한 값만 담았다.

> **구성**
> 본편 1–7부는 현행 비전 RL(Ray+IR 교사 직결)이다. 그 바탕인 블라인드 보행의 개념·용어·CENet·AdaBoot·데이터수집·GAE·PPO·대칭 증강·배포 계약·실험 구성은 **기초편 A0–A5**에 함께 실었다(원래 `gd_lab/gd_lab_training_v2.pdf`). 처음 읽는 사람은 1부 → 기초편 A0–A3 → 2부 이후 순서를 권한다. 2026-09 초기 비전 설계(렌더 카메라 학생 80×45)는 현행이 아니다.

1부. 목적과 전체 구조
1.1 목적 · 1.2 방법의 변천 · 1.3 현행 구조 한 장 요약
2부. 입출력 구조
2.1 Actor 입력 · 2.2 지형 프레임과 기억 · 2.3 출력 · 2.4 Critic
3부. 지형처리기
3.1 학습 쪽: Ray+IR 관측 · 3.2 배포 쪽: depth+IR 매퍼 · 3.3 버전 이력 · 3.4 알려진 오차
4부. Actor-Critic·PPO 학습
4.1 네트워크 · 4.2 초기화 · 4.3 환경 · 4.4 보상 · 4.5 PPO와 안정성 · 4.6 모델 선택 · 4.7 파인튜(aug149) · 4.8 학생(보류)
5부. 평가 도구
5.1 E1 · 5.2 RL Replay · 5.3 MuJoCo 배포 검증
6부. 배포
6.1 export · 6.2 번들·계약 · 6.3 Pilot 런타임 · 6.4 한계
7부. 현재 결과와 과제 · **부록** 코드 위치
기초편 A. 블라인드 보행 공통 기반
A0 개념·용어·규모·3대 신경망 · A1 액션(Actor 입력·CENet·AdaBoot·PD) · A2 데이터수집(관측·롤아웃·보상·커리큘럼·지형·랜덤화) · A3 학습(텐서 매핑·GAE·PPO 손실·CENet 손실·KL LR·대칭 증강) · A4 배포(계약·export·백엔드) · A5 실험(TRAIN_ARM·성능)

1부. 목적과 전체 구조

1.1 목적

RBQ10 안내견 로봇이 **시각장애인을 하네스로 이끌며** 지하철 승강장 갭(2–26 cm, 단차 0–16 cm), 계단(5–25 cm), 경사·거친 바닥을 넘도록 한다. 블라인드 정책은 갭 앞에서 발이 빠지고 계단 앞에서 걸려 넘어진다 — 몸 앞·아래 지형을 보는 정책이 필요하다. 추가 요구:

- **페이로드** 0/6 kg(실사양 5 kg: 머리 2 + 중앙 2.5 + 후방 0.5)에서 같은 성능.
- **핸들 외력**: 계단에서 사람이 하네스를 당기거나 미는 힘(오르막 ≤200 N, 내리막 ≤150 N)을 견딘다.
- **실제 센서**: 로봇 하부 RealSense D4xx 4대(depth + IR). LiDAR는 내년 탑재 예정이며, 지금 구조는 LiDAR 경로도 같은 지도 형식으로 받는다.
- **실시간**: 정책 100 Hz, 지형 캡처 10 Hz, 로봇 CPU에서 ONNX 추론.

1.2 방법의 변천 — 왜 "교사 직결"인가

세대	방식	결과 / 버린 이유
CVTT (09-23~10-01)	렌더 카메라 가시성 교사 + 카메라 학생(RVLD/GAVD)	학생이 갭·계단 미통과. 렌더 비용 큼
BIVT-Ray (09-30~10-04)	블라인드 시작 + 레이캐스트 가시성 높이맵 교사	교사는 좋아졌지만 카메라 학생 증류가 병목
GAST v1 → v2.1 (10-01~10-13)	시간 기억(8프레임, pose warp) 지형 encoder	v1 NaN 실패, v2.1(math SDPA)로 해결. 학생 B3 E1 72.6%가 상한
Ray+IR (10-14~)	교사 입력 자체를 **배포 매퍼가 만들 수 있는 지도**(레이캐스트 가시 셀 + 노이즈 + IR)로 학습	학생 없이 매퍼 지도를 교사에 직접 넣음. E1 카메라+필터 갭 96.2%(3906)

> **핵심 결정**
>
> 카메라 영상 → latent를 배우는 학생 대신, **센서 → 지형처리기(매퍼) → 11×17 지도**를 사람이 만든 처리기로 고정하고, 교사가 그 지도 형식(가시 셀만, 노이즈·결측·blackout 포함)으로 직접 학습하도록 했다. 증류 손실이 사라지고, 센서가 바뀌어도 처리기만 바꾸면 된다.

1.3 현행 구조 한 장 요약

```
[학습: Isaac Lab, gd_lab_vrl]
 GT 지형 메시 → RaycastIrTerrain (4카메라 가시성 · 노이즈 · dropout · blackout · IR 프록시)
   → 8프레임 기억 [h, valid, IR] × 187 × 8 → GAST encoder → terrain latent 32 ─┐
 고유감각 46 × 5 → CENet → z_t 19 ──────────────────────────────────────────────┼→ actor → 관절 12 (100 Hz)
 고유감각 46 ──────────────────────────────────────────────────────────────────┘
 critic (특권: 전체 높이맵 · 접촉 · 마찰 · 질량 …) → PPO

[배포: gd_rbq10_deploy_vrl]
 D4xx × 4 (depth + IR) → camfilter136 → L0S 매퍼 (v25or) → 11×17 [h, valid, IR] 프레임, 10 Hz
   → UDP GSC2 → Pilot: 49슬롯 ring → 8프레임 → gast_encoder.onnx → policy_vrl.onnx → PD
```

기초편 A. 블라인드 보행 공통 기반 (gd_lab 학습 파이프라인 총정리 v2에서 옮김)

비전 교사는 블라인드 DreamWaQ 구조 위에 지형 경로를 붙인 것이다. 아래 A0–A5는 `gd_lab/docs/gd_lab_training_v2.html`(2026-09-20, gd_lab main 50b848b)의 개념·용어·액션·데이터수집·학습·배포·실험 내용을 그대로 옮겼다(원문 6부 "비전 RL 확장"은 현행이 아니어서 제외). 원문의 수치는 **블라인드 ARM1(50 Hz) 기준**이다. 비전 교사(3906)와 다른 값은 아래 표에 모았고, 본편(2–7부)이 현행 값이다.

항목	기초편 A (블라인드 ARM1, 09-20)	비전 교사 3906 (현행)
정책 주기	sim dt 0.005 × decimation 4 = 0.02 s (50 Hz), 에피소드 1000 step	decimation 2 = 0.01 s (**100 Hz, ARM4**), 에피소드 20 s = 2000 step
PD 게인	ARM1 현재 게인 hip 88.14 / knee 102.22, kd 1.99	ARM4 nominal hip 123.39 / knee 127.77, kd 2.4 (A5.1의 ARM 4)
Actor 입력	46 × 4프레임 + CENet code 19 = 203	203 + terrain latent 32 = **235** (2.1절)
Critic 입력	298	298 (같은 특권 관측, 높이맵 포함)
병렬 env	4096	1024 (3906 학습), DDP 시 `--total_envs`
지형	10종	11종, **platform_gap 50%** (4.3절)
보상	21항목	v2.1 기본 + 갭·계단·핸들 외란 항목 (4.4절)
학습률	adaptive KL, 하한 3e-5	fixed: policy 1e-4 / terrain 1e-3 (warm start 설정)
추가 신경망	Actor · Critic · CENet	+ GAST 지형 encoder(시간 기억) + 재구성 decoder (4.1절)
배포	camel.policy.v1 계약 (Meta 백엔드)	GAST 전용 2-ONNX 번들(actor + encoder) + manifest, 계약 밖 VRL 백엔드 (6부)

A0. 개념 — 전체 그림

A0.1 gd_lab이 하는 일

IsaacLab(시뮬레이터) + `rsl-rl-lib 3.0.1`(PPO 구현) 위에 DreamWaQ 스타일 blind 사족보행 정책을 학습시킨다. **blind**란 배포되는 Actor가 시각·거리 센서 없이 자기수용감각만으로 걷는다는 뜻이고, 학습 중에만 존재하는 Critic은 지형 높이·접촉력 등 특권정보를 추가로 본다. 이 저장소의 독자 설계는 커리큘럼이 아니라 **CENet**(맥락 추정 네트워크)과 **AdaBoot**(적응형 부트스트랩)이다.

> **왜 Critic만 특권정보를 보는가 — 비대칭 Actor-Critic**
> Critic은 **학습 중에만** 쓰이고 로봇에 실리지 않는다. 그러니 실기에 없는 정보(지형 높이맵, 마찰계수, 접촉력)를 마음껏 줘도 배포에 문제가 없다. 가치추정이 정확해지면 advantage가 정확해지고, 그 결과 Actor의 경사가 덜 흔들린다. "배포될 쪽만 제약을 지키면 된다"는 것이 비대칭 구조의 요지다.

A0.2 용어 — 스텝 / 에피소드 / 이터레이션 / 배치

개념	정의
스텝(step)	정책이 1번 행동하는 단위. `sim.dt`(0.005 s) × `decimation`(4) = **0.02 s (50 Hz)**. 물리는 200 Hz로 돌지만 정책 출력은 4틱 동안 유지된다
에피소드	로봇 1마리의 리셋~종료(낙상 또는 타임아웃). `episode_length_s`=20 s = 최대 1000스텝
`num_envs`	병렬 로봇 수(4096). GPU 병렬도일 뿐 난이도와 무관
iteration	(전 env가 `num_steps_per_env`=100스텝 수집) + (5 epoch × 4 minibatch 갱신) 1사이클. 정책은 100스텝 내내 **고정**, 끝난 뒤 한 번만 갱신
배치	4096 × 100 = **409,600** transition / iteration

> **1 iteration ≠ 1 episode**
> 100스텝 iteration과 최대 1000스텝 episode는 단위가 다르다. 넘어지지 않으면 한 에피소드가 약 10 iteration에 걸쳐 이어진다(v1의 48스텝 기준으로는 20.8이었다). 4096마리의 리셋 시점은 완전히 비동기라서, 어느 iteration에도 "막 리셋된 로봇"과 "900스텝째 걷는 로봇"이 섞여 있다.

A0.3 학습 규모 파라미터

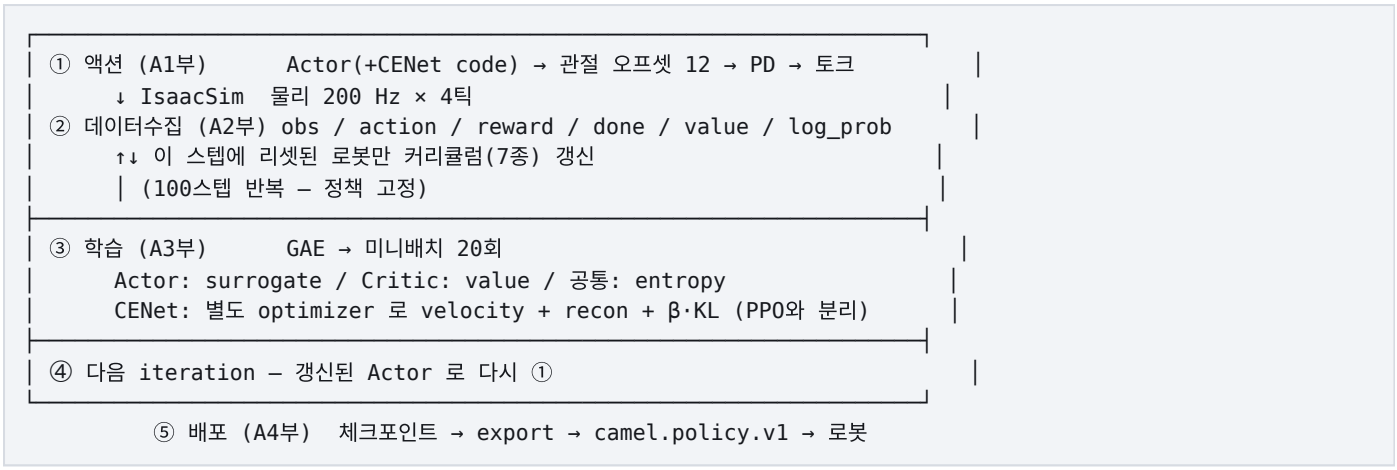
항목	값	비고
`num_envs`	4096	play 태스크는 축소
`num_steps_per_env`	100	롤아웃 horizon (v1: 48, 벤더: 60)
`num_learning_epochs`	5	롤아웃 1회당 반복 학습
`num_mini_batches`	4	→ iteration당 경사 갱신 20회
`max_iterations`	40000	4-arm 실험 기준 (기본 cfg 50000)
`save_interval`	1000	체크포인트 주기
`episode_length_s`	20.0 s	= 1000 스텝
`seed`	42	4-arm 고정

`num_envs`를 늘리면 iteration당 데이터는 늘지만 iteration 자체도 길어져서 총 학습시간이 오히려 늘 수 있다(v1 실측: 4096→8192에서 2.35 s→2.90 s). 학습시간을 줄이는 레버는 `num_envs`가 아니라 `max_iterations`다.

A0.4 3대 신경망

	Actor	Critic	CENet
정규화	EmpiricalNormalization(230)	EmpiricalNormalization(298)	Actor 정규화기 공유
입력	46×4 + code 19 = **203**	298	230 (46×5, term-major)
은닉	203→512→256→128 (ELU)	298→512→256→128 (ELU)	encoder 230→128→64 (ELU)
출력	128→12, 활성화 없음 = μ	128→1 = V(s)	head 3개: 64→16(mean) / 64→16(logvar) / 64→3(velocity) decoder 19→64→128→46
분포	Normal(μ, σ), σ는 학습되는 스칼라 1개 (`init_noise_std`=1.0)	Actor와 몸통 공유 없음	배포 시 decoder 출력은 버려진다

A0.5 전체 파이프라인


① 액션 (A1부) Actor(+CENet code) → 관절 오프셋 12 → PD → 토크
↓ IsaacSim 물리 200 Hz × 4틱
② 데이터수집 (A2부) obs / action / reward / done / value / log_prob
↑↓ 이 스텝에 리셋된 로봇만 커리큘럼(7종) 갱신
(100스텝 반복 — 정책 고정)
③ 학습 (A3부) GAE → 미니배치 20회
Actor: surrogate / Critic: value / 공통: entropy
CENet: 별도 optimizer 로 velocity + recon + β·KL (PPO와 분리)
④ 다음 iteration — 갱신된 Actor 로 다시 ①
⑤ 배포 (A4부) 체크포인트 → export → camel.policy.v1 → 로봇

A1. 액션 — Actor가 무엇을 어떻게 산출하는가

A1.1 Actor 입력 203

v1에서는 최신 1프레임(45) + code(19) = 64였다. 지금은 `actor_history_steps=4`로 **최근 4프레임을 최신순**으로 받는다.

```
actor_input = concat( frame[t], frame[t−1], frame[t−2], frame[t−3], code )
  = 46 × 4 + 19 = 203

code = concat( velocity.detach(), z ) = 3 + 16 = 19
```

정규화된 history에서 `frames_idx`로 뽑는다. history는 **term-major**로 저장되므로 (항목별 5스텝 뭉치를 이어붙인 배치), 특정 시점의 한 프레임을 꺼내려면 항목마다 흩어진 위치를 모아야 한다. 그 인덱스를 `__init__`에서 한 번 계산해 버퍼로 등록해 둔다.

```
term-major 배치 (46 × 5 = 230)
  [ang_vel  t-4 t-3 t-2 t-1 t] [gravity t-4 … t] [cmd …] [joint_pos …] [joint_vel …] [actions …] [payload
   └ 3×5=15 ┘                  └ 15 ┘           15       60            60            60          5

frames_idx = 각 term 의 "t" 위치 → 그다음 "t-1" 위치 → … (최신 먼저, K=4)
latest_idx = frames_idx[:46]  (최신 1프레임만)
```

> **왜 계약은 바뀌지 않는가, 그리고 왜 체크포인트는 깨지는가**
>
> 4프레임은 이미 저장 중인 5프레임 안에 들어가므로 **관측 규격과 ONNX 계약은 그대로**다. 반면 Actor 1층의 입력 폭이 `K×46+19`이므로 **K를 바꾸면 체크포인트를 이어받을 수 없다**. K=4가 K=1보다 낫다는 실험 근거는 저장소에 없다(4-arm 문서도 K를 다루지 않는다) — 현재로서는 "blind 기준선과 맞춘 값"으로 이해하는 것이 정확하다.

A1.2 CENet — 맥락 추정 네트워크

Actor는 자기 몸의 상태만 본다. 그런데 "지금 밟고 있는 바닥이 미끄러운가", "짐이 실려 무거운가" 같은 **관측되지 않는 맥락**은 걸음을 크게 좌우한다. CENet은 관측 **히스토리**에서 그 맥락을 추정해 잠재벡터로 압축하고, 동시에 몸통 선속도를 회귀한다.

```
  obs_history 230        encoder(공유 몸통)            mean_latent   Linear(64→16)  ┐ 정답 없음
  (46×5, term-major) ─→  230→128→64 (ELU) ─┬─→                                     │ (자기지도)
                              feat(64)     ├─→  logvar_latent Linear(64→16)        ┘ ±10 clamp
                                           └─→  mean_velocity Linear(64→3)  ← 지도학습(정답 있음)

   z 샘플링   학습: z = mean + randn × exp(0.5 · logvar)     ← 재매개변수화 트릭
              추론: z = mean_latent                            ← 결정론적

   code = concat( velocity.detach(), z )  19차원  →  Actor 입력 / decoder 입력
   decoder 19→64→128(ELU)→46  →  o(t+1) 재구성 시도
```

세 개의 헤드

"헤드"란 비싼 특징추출(encoder)을 한 번만 하고 그 결과 `feat`를 재사용해 각자 다른 걸 내는 얇은 마지막 층이다. 여기서는 셋 다 `Linear` 한 층씩이다.

헤드	출력	학습 신호
`mean_velocity`	3	**지도학습**. Critic 관측의 진짜 `base_lin_vel`과 MSE
`mean_latent`	16	**자기지도**. 재구성 손실 + KL로만 형태가 잡힘
`logvar_latent`	16	불확실성. ±10으로 clamp — 없으면 `exp()`가 발산

> **재매개변수화 트릭이 필요한 이유**
>
> z를 분포에서 그냥 뽑으면 그 표집 연산에 경사가 흐르지 않아 encoder를 학습시킬 수 없다. `z = mean + randn × exp(0.5·logvar)`로 쓰면 무작위성이 randn(경사 불필요)에 격리되고, mean과 logvar는 곱셈·덧셈으로 연결돼 경사가 그대로 통과한다. 추론 시에는 표집을 끄고 `z = mean`을 쓴다 — 배포에서 같은 입력에 같은 출력이 나와야 하기 때문이다.

`velocity.detach()`에 주목. code에 들어가는 velocity는 **경사를 끊어서** 넣는다. Actor가 "속도 추정을 자기에게 편한 방향으로 왜곡"하지 못하게 하려는 것이다. 속도 헤드는 오직 진짜 속도와의 MSE로만 학습된다.

A1.3 AdaBoot — 속도를 참값으로 줄지 추정값으로 줄지

학습 초기에는 CENet의 속도 추정이 부정확하다. 그 값을 그대로 Actor에 넣으면 정책이 잘못된 속도 인식 위에 올라선다. AdaBoot는 **동전을 던져** 일정 확률로 Critic 관측의 진짜 `base_lin_vel`을 대신 주입한다.

```
CV = std(에피소드 총보상) / |mean(에피소드 총보상)|   (변동계수)
p_boot = 1 − tanh(CV)   ← 추정을 믿을 확률
p_boot ← ema × p_boot_prev + (1−ema) × p_boot_new,  ema = 0.8
```

성능이 들쭉날쭉하면(CV 큼) `tanh(CV)`가 1에 가까워져 p_boot이 0으로 내려가고 → **참값을 더 자주 주입**한다. 학습이 안정되면 CV가 작아져 p_boot이 1에 가까워지고 → 자기 추정을 쓴다.

파라미터	값	의미
`adaboot_enabled`	true	on/off
`adaboot_min_updates`	10	이 횟수 전에는 통계를 믿지 않음
`adaboot_ema`	0.8	p_boot의 지수이동평균 계수

> **동전은 iteration당 한 번만 던진다**
>
> `notify_iteration_done()`이 플래그를 세우고, 다음 롤아웃의 **첫 act**에서 한 번 재표집한다. 롤아웃 중간에 바뀌면 같은 배치 안에 "참값을 본 transition"과 "추정값을 본 transition"이 섞여서, PPO가 일관되지 않은 입력 규약으로 학습하게 된다.

A1.4 액션 → PD 제어 → 시뮬레이션

```
target_joint_pos = default_joint_pos + 0.25 × action   (action_scale = 0.25)
τ = kp × (target − q) − kd × q̇
```

단계	내용
액션 의미	관절 오프셋 12차원(절대각 아님). 활성화 없음 — 무한대 범위지만 clamp 5.0
하드리밋	관절 한계 5° 이내 진입 시 saturation 블렌딩(`JointPositionActionWithLimitCfg`)
PD 게인	드라이브 게인 **양자의 정수배**로만 지정. `RBQ_DRIVE_KP_STEP` = {hip 17.62734, knee 25.55443} 기본(ARM 1·2): hip 5 step = **88.1367**, knee 4 step = **102.2177**, kd ≈ 1.99 nominal(ARM 3·4): hip 7 step = **123.39**, knee 5 step = **127.77**, kd 2.4
게인 랜덤화	리셋마다 기준값의 0.85~1.15배

게인을 양자 정수배로만 쓰는 이유는 실제 모터 드라이버가 이산적인 게인 설정만 받기 때문이다 — 학습이 하드웨어가 구현할 수 없는 값을 쓰지 않게 한다.

게인은 배포에 그대로 실린다

학습에 쓴 게인이 ONNX 메타데이터에 기록되어 로봇 액추에이터로 전달된다(A4부). 즉 게인 선택은 시뮬 내부 설정이 아니라 **실기에 박히는 값**이고, 4-arm 실험이 이것을 축으로 삼은 이유다.

A2. 데이터수집 — 무엇이, 어떤 난이도에서 모이는가

A2.1 관측 항목

Policy 관측 — 1-step 46 × history 5 = 230

항목	차원	노이즈	물리적 의미
base_ang_vel	3	U(−0.2, 0.2), scale 0.25	몸통 각속도
projected_gravity	3	U(−0.05, 0.05)	중력벡터의 몸체좌표 투영 — 자세
velocity_commands	3	없음	명령 (v_x, v_y, ω_z)
joint_pos	12	U(−0.01, 0.01)	기본자세 대비 관절 위치
joint_vel	12	U(−1.5, 1.5), scale 0.05	관절 속도
actions	12	없음	직전 액션
payload	1	**없음**	운용자 입력 하중 × 0.2. 추정이 아니라 알려진 값이라 노이즈를 주지 않는다

payload는 proprio 블록 **뒤에** 선언된다 — 앞 45차원이 배포 계약상 안정된 접두사이므로, 이 항을 켜고 꺼도 그 순서가 흔들리지 않는다.

Critic 관측 — 298

앞 45는 policy와 동일한 proprio(노이즈 없는 깨끗한 값)이고, 그 뒤에 특권정보가 붙는다.

추가 항목	차원	의미
base_lin_vel	3	실제 선속도 — **CENet 속도 헤드의 정답**
feet_around_height_from_terrain	16	발 4개 × 2×2 ray, 지형 대비 발 높이
feet_contact	4	발별 접촉 이진값
feet_contact_forces	12	발별 접촉력 벡터
friction_coeff	2	(정적, 동적) 마찰계수
base_mass_offset	1	도메인 랜덤화 질량 오프셋
actuator_gain_scale	24	PD 게인 스케일(kp×12, kd×12)
push_delta_v	3	외란 push가 준 속도 변화량
height_scan	187	지형 스캔 그리드 GridPatternCfg 17×11, clip ±1, scale 5
payload	1	policy와 동일

```
배치:  45 + 3 + 16 + 4 + 12 + 2 + 1 + 24 + 3 + 187 + 1 = 298
       base_lin_vel  구간 [45, 48)      ← CENet 타깃 슬라이스
       height_scan   구간 [110, 297)    ← 비전 teacher 의 terrain encoder 입력
       payload       구간 [297, 298)
```

> **height_scan은 더 이상 마지막이 아니다**
>
> v1에서는 height_scan이 Critic 관측의 끝이라 "offset부터 끝까지"로 잘라 쓰는 코드가 있었다. 지금은 뒤에 payload가 붙어서 그렇게 자르면 188개를 가져오고 11×17 격자 재구성이 깨진다. 반드시 **시작·끝을 모두 지정**해야 한다.

A2.2 롤아웃 — 100스텝 × 4096로봇

분류	생성 주체	수집 텐서
관측	ObservationManager	policy (100, 4096, 230) / critic (100, 4096, 298)
액션	Actor	(100, 4096, 12)
다음 관측	IsaacSim(물리 후 재계산)	(100, 4096, 46) — CENet 재구성 타깃용 최신 프레임만
종료	TerminationManager	(100, 4096, 1) + `time_outs`(별도)
리워드	RewardManager(21항목 × weight)	(100, 4096, 1)
가치	Critic	(100, 4096, 1)
μ, σ, log_prob	Actor 분포	PPO ratio 계산용

환경이 "관측·피드백·리워드"를, 정책이 "액션·확률정보"를 만들고, `RolloutStorage`가 전부를 (100, 4096, ·) 텐서로 모은다. 학습은 이 창고가 다 찬 뒤에 시작된다.

A2.3 리워드 21항목

총 보상은 항목별 값 × weight의 합이고, 각 항목은 스텝마다 계산된 뒤 `step_dt`가 곱해진다.

항목	weight	정의 / 물리적 의미
추종 (양수)		
track_lin_vel_xy_exp	+1.0	$\exp(-\lVert \mathrm{cmd}_{xy} - v_{xy} \rVert^2 / \sigma)$ — 명령 선속도 추종
track_ang_vel_z_exp	+0.8	$\exp(-(\mathrm{cmd}_{\omega} - \omega_z)^2 / \sigma)$ (v1: 0.5)
자세·안정		
lin_vel_z_l2	−2.0	v_z^2 — 수직 출렁임 억제
ang_vel_xy_l2	−0.15	$\omega_x^2+\omega_y^2$ — roll/pitch 각속도 억제
base_height	−15.0	$(z - (0.52 + \mathrm{mean}(\mathrm{ray_z})))^2$ — **지형 상대** 몸통높이 유지
flat_orientation_roll_l2	−2.0	g_y^2 — 롤은 지형과 무관하게 항상 flat
flat_orientation_pitch_l2	−1.0	피치는 **지형 경사에 정렬된 목표**와 비교 — 경사로에서 몸을 기울여도 벌하지 않는다
보행 품질		
feet_cadence_overrun	−1.0	스윙 0.4 s / 스탠스 0.6 s **초과분만** 과금(단측) (v1: 0.3/0.4)
foot_clearance	−1.0	스윙 중 지형 대비 높이 미달분². 계단 라이저에서 요구치가 자동으로 올라간다
feet_slipping	−0.1	접촉 중 발 수평속도
feet_touchdown	−1.0	착지 순간 발 속도 — 충격 완화 (v1: −0.5)
joint_deviation_hip_l1	−0.3	고관절 외전 $\lvert\Delta q\rvert$ — 보폭 폭을 기본 발자국 안에 유지
정지 명령		
stand_still	−0.5	명령 ≈0일 때 기본자세 이탈 L1
stand_still_feet_contact	−1.0	정지 시 네 발 접지 유지 — 서 있을 때 발을 들지 않게
액추에이터 부담		
dof_torques_l2	−2.5e−5	$\Sigma\tau^2$
joint_power	−2e−5	$\Sigma\lvert\tau \cdot \dot{q}\rvert$ — 에너지 소모를 직접 과금(토크²와 달리 방향 고려)
dof_acc_l2	−2.5e−7	$\Sigma\ddot{q}^2$
dof_vel	−5e−4	$\Sigma\dot{q}^2$
dof_pos_limits	−2.0	관절 한계 초과분. **커리큘럼 램프 제외** — 항상 풀 weight
action_rate_l2	−0.02	$\lVert a_t - a_{t-1} \rVert^2$
smoothness	−0.02	$\lVert a_t - 2a_{t-1} + a_{t-2} \rVert^2$ (jerk). 리셋 직후 2스텝 면제
안전		
undesired_contacts	−1.0	허벅지·종아리 접촉력 >1 N. **몸통은 제외** — 낙상은 종료로 이미 과금되므로 이중 과금이 된다
termination_penalty	−2.0	낙상 종료 자체. **v1에서는 0.0(비활성)**

> **가중치가 크다고 지배적인 것은 아니다**
>
> base_height는 −15.0으로 최대 가중치지만 실측 기여도는 21항목 중 하위권이다 (ARM 1: Episode_Reward −0.018 vs 1위 track_lin_vel_xy_exp +0.695). 정책이 이미 높이를 RMS 3.8 cm 이내로 지켜 제곱항이 작기 때문이다. **weight는 "얼마나 비싼가", 실현 기여도는 거기에 "실제로 얼마나 틀리는가"를 곱한 값**이다.

> **base_height의 숨은 함정**
>
> 기준면이 `mean(ray_z)` — 몸통 아래 1.6 m × 1.0 m 187개 레이의 **평균** 지면 높이다. 계단 모서리를 걸치면 이 평균이 아랫단과 윗단의 혼합값이 되어, 윗단에 서 있는데도 "더 낮게 앉아라"는 신호가 나온다. 일정한 경사로에서는 평균이 몸통 중심 높이와 거의 같아 이 편향이 생기지 않는다.

A2.4 커리큘럼 7종

커리큘럼은 리워드 **공식**을 바꾸지 않는다. 공식이 받는 **입력**(환경·명령)을 바꾸거나, 공식에 곱해지는 **weight**를 바꾼다. 갱신은 **리셋된 env에 대해서만** 일어난다.

#	이름	층위	바꾸는 것
1	`terrain_levels_vel_cmd_aware`	환경(World)	지형 난이도(행). **업스트림이 아니라 자체 구현** — `min_moving_time=2.0`초 이상 실제로 움직인 env만 승급 판정 대상. 서 있으라는 명령을 받은 env가 "못 걸었다"고 강등되는 것을 막는다
2	`command_levels_lin_vel`	과제(Task)	명령 선속도 샘플링 범위
3	`command_levels_ang_vel`	과제(Task)	명령 각속도 샘플링 범위
4	`penalty_terrain_schedule`	평가 기준	6개 리워드 weight 배수 κ (0.5 → 1.0). 초반에 페널티를 절반으로 깎아 탐색을 허용
5	`pulse_prob_schedule`	과제(Task)	명령 급변(pulse) 발생 확률. **4-arm 실험에서는 0으로 꺼둔다**
6	`terrain_levels_per_family`	모니터링	지형 종류별 평균 레벨. 값을 바꾸지 않고 기록만 한다
7	`termination_per_family`	모니터링	지형 종류별 낙상 종료율

지형 커리큘럼의 승급·강등 규칙

`move_up` = 이동거리 > `terrain.size[0] / 2` = **4.0 m**
`move_down` = 이동거리 < $\|\text{cmd}_{xy}\|$ × `episode_length_s` × 0.5 (그리고 `move_up`이 아닐 때)
최고 레벨(15) 초과 시 → **0~14 중 무작위 레벨로 재배치**

> **6·7번이 왜 필요했는가**
>
> IsaacLab의 커리큘럼은 env마다 **레벨(행) 하나**만 추적한다. 지형 종류(열)는 `arange(num_envs) // (num_envs/num_cols)`로 초기에 결정론적으로 배정된 뒤 **학습 내내 바뀌지 않는다**. 그래서 전체 평균 레벨만 보면 *"특정 지형이 4레벨에 멈춰 있는데 평균은 9"*인 상황이 보이지 않는다. 6·7번이 그 사각지대를 없앤다.

> **실패율은 0으로 수렴하지 않는다 — 설계상 그렇다**
>
> 승급 조건이 "4 m를 갔는가"이므로, 커리큘럼은 **실패할 때까지 계속 난이도를 올리는 자기균형 장치**다. 실패율은 최소화 대상이 아니라 **제어 신호**이고, 평형은 0이 아닌 값에 잡힌다. 게다가 최고 레벨 도달 시 무작위 재배치가 있어 평균 레벨이 최대치로 수렴할 수도 없다 — 정복한 지형의 env도 계속 아래로 떨어졌다 다시 올라온다.

A2.5 지형 10종

격자는 **12열(종류) × 15행(난이도)** = 180타일, 타일 하나 8 m × 8 m, 테두리 30 m. 행 r의 난이도는 `(r + U(0,1)) / 15`로 결정되어, 같은 행이라도 열마다 값이 조금씩 다르다.

종류	proportion	파라미터
pyramid_stairs	0.1	step_width 0.25, height 0.05~0.20
pyramid_stairs_wide	0.1	step_width 0.30, height 0.05~**0.25**
pyramid_stairs_inv	0.1	step_width 0.25, height 0.05~0.20
pyramid_stairs_inv_wide	0.1	step_width 0.30, height 0.05~**0.25**
pyramid_stairs_nose	0.1	노징(오버행) 깊이·두께 3 cm — **이 저장소 고유**, 벤더에 없음
pyramid_stairs_inv_nose	0.1	동상
ring_fence	0.1	동심원 4겹 턱, 두께 0.06, 높이 0.03~0.15
random_rough	0.1	noise 0.01~0.10
hf_pyramid_slope	0.1	slope 0.1~0.5 (최대 26.6°)
hf_pyramid_slope_inv	0.1	slope 0.1~0.5

> **proportion과 실제 배정은 다르다**
> 12열에 10종을 누적합으로 나누므로 양자화 오차가 생긴다. 설정은 전부 0.1(=10%)이지만 실제로는 일부 종류가 2열(16.7%), 나머지가 1열(8.3%)을 받는다. 지형별 통계를 해석할 때 이 불균형을 감안해야 한다. 열 수를 10의 배수로 두면 사라지는 문제다.

env는 열에 결정론적으로 배정되고 초기 레벨만 무작위인데, 그것도 max_init_terrain_level=3이라 **0~3행에서만 시작**한다. 높은 레벨에서 시작하면 학습 안 된 정책이 바로 자빠져 회복하지 못하기 때문이다.

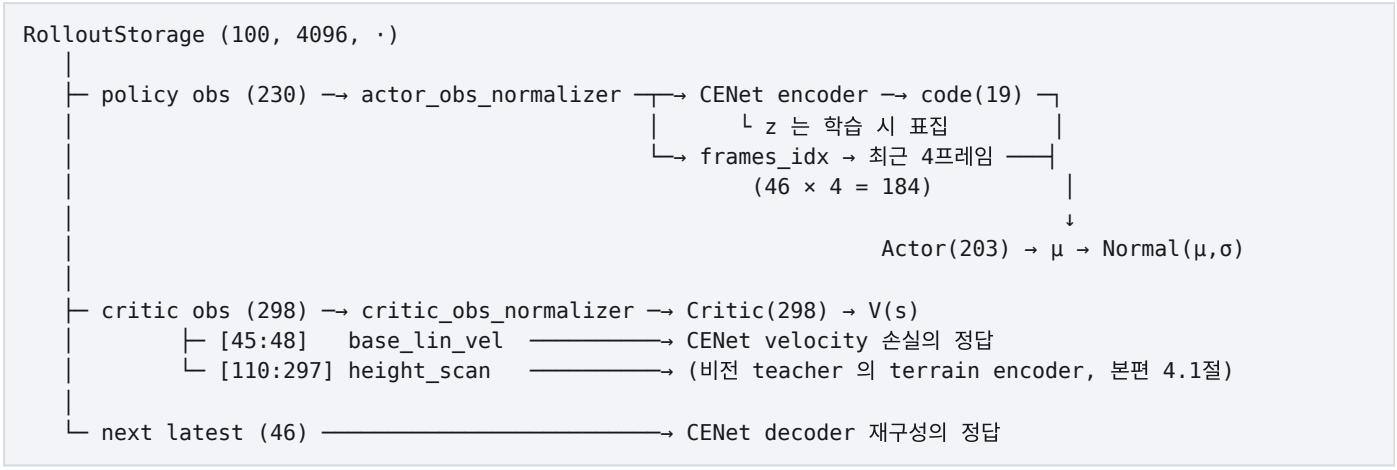
A2.6 도메인 랜덤화 — 무엇을 흔드는가

이벤트	시점	범위
마찰계수	startup	정적/동적, 64 버킷
add_base_mass	startup	trunk ±2.0 kg
base_com	startup	x,y ±0.05 m / z ±0.01 m
관성 대각	startup	IsaacLab이 안 흔드는 항 — 질량만 바꾸면 회전 응답이 그대로 남아서
randomize_actuator_gains	reset	kp·kd 각각 0.85~1.15배
randomize_payload	reset	**0 kg 20% / 6 kg 80%**, 위치 x ∈ [−0.05, 0.05] m, y=0, z=0.07 m
reset_base	reset	x,y ±0.5 m, yaw ±π
push	interval	명령 위상과 무관하게 — 특정 국면에 묶이면 그 국면만 대비하게 된다

payload는 점질량으로 질량·CoM·관성을 평행축 정리로 재계산한다(core/payload_math.py). 장착 **위치**는 무작위지만 정책에게 알려주지 않는다 — 관측은 질량 스칼라 하나뿐이고, 위치 변동은 proprioception으로 감당해야 하는 외란으로 남긴 설계다.

A3. 학습 — 모은 데이터로 무엇을 하는가

A3.1 롤아웃 → 신경망 텐서 매핑



정규화기(`EmpiricalNormalization`)는 관측의 이동평균·분산을 계속 갱신한다. CENet의 재구성 타깃도 **정규화된 공간**에서 비교해야 한다 — 원시값은 범위가 제각각이라 특정 항목이 손실을 독점한다.

A3.2 GAE — advantage와 return

먼저 리턴과 가치의 정의:

$$G_t = \Sigma_{k\geq 0}\ \gamma^k\ r_{t+k} \quad \text{(할인 누적보상)}$$
$$V(s_t) \approx E[\ G_t \mid s_t\ ] \quad \text{(Critic이 근사하는 값)}$$

한 스텝의 예측 오차(TD error)와 그것을 지수가중으로 누적한 GAE:

$$\delta_t = r_t + \gamma\ V(s_{t+1}) - V(s_t)$$
$$\hat{A}_t^{GAE(\gamma,\lambda)} = \Sigma_{l\geq 0}\ (\gamma\lambda)^l\ \delta_{t+l}$$

$$\gamma = 0.99 \quad \lambda = 0.95 \quad \rightarrow \quad return_t = \hat{A}_t + V(s_t)$$

λ는 **편향-분산 손잡이**다. λ=0이면 δ_t 하나만 써서 분산은 작지만 Critic 오차에 직접 휘둘린다(편향 큼). λ=1이면 몬테카를로에 가까워 편향은 없지만 분산이 크다. 0.95는 그 사이다.

> **타임아웃은 실패가 아니다**
> 에피소드가 `time_outs`(20초 경과)로 끝난 것은 **실패가 아니라 관측 중단**이다. 그 시점의 미래 가치는 실제로 존재하므로 `V(s_last)`로 부트스트랩해서 보정한다. 반면 낙상 종료는 부트스트랩하지 않는다 — 넘어진 상태의 미래 가치는 정말로 없다. 이 구분을 놓치면 "20초를 버틴 좋은 에피소드"가 "넘어진 에피소드"와 같은 취급을 받는다.

A3.3 PPO 손실

세 항의 합을 최소화한다. r은 정책 확률비, Â는 정규화된 advantage다.

$$r_t(\theta) = \pi_\theta(a_t|s_t) \;/\; \pi_{\theta_old}(a_t|s_t)$$

$$L^{surrogate} = -E[\; \min(\; r_t\hat{A}_t,\;\; clip(r_t,\; 1-\varepsilon,\; 1+\varepsilon)\cdot\hat{A}_t \;)\;] \quad \varepsilon = 0.2$$

$$L^{value} = E[\; \max(\; (V-G)^2,\; (V_{clip}-G)^2 \;)\;] \quad \text{(clipped value loss)}$$

$$L^{entropy} = -E[\; H(\pi) \;]$$

$$\mathbf{L = L^{surrogate} + 1.0 \cdot L^{value} + 0.01 \cdot L^{entropy}}$$

항	계수	역할
surrogate (clip)	ε = 0.2	확률비를 [0.8, 1.2]로 잘라 한 번에 너무 멀리 가는 것을 차단
value loss	1.0	Critic 회귀. clip을 쓰는 이유는 가치 추정이 한 번에 크게 튀는 것을 막기 위함
entropy	0.01	탐색 유지 — 실질적으로 **액션 노이즈 σ의 평형값**을 정한다
`max_grad_norm`	1.0	경사 노름 클리핑

> **entropy 계수의 물리적 의미**
>
> 대각 가우시안의 엔트로피는 $H = \Sigma \log \sigma_i$ + 상수이므로 $\partial H / \partial \log \sigma = 1$이다. 즉 이 항은 **매 갱신 log σ를 계수 크기만큼 일정하게 밀어올리는 힘**이고, surrogate loss가 노이즈가 성능을 깎는 만큼 반대로 당긴다. σ는 그 평형점에 정착한다.
>
> ARM 1 실측 σ ≈ 0.33. `action_scale 0.25`를 곱하면 관절 목표각에 1σ ≈ 4.7°이고, 다리 길이 0.4 m 기준 **발 착지 위치가 약 ±3 cm 흩어진다**. 14 cm 단차나 계단 노징 앞에서 결코 작지 않은 산포다.

> **base_contact은 노이즈가 섞인 값이다**
>
> 로그의 `Episode_Termination/base_contact`은 **탐색 노이즈가 들어간 rollout**에서 측정된다. 배포되는 결정론적 정책(`act_inference`, σ 미적용)의 실패율은 이보다 낮다. 두 수치를 직접 비교하면 안 된다 — v1에서 결정론적 진단이 12%, 같은 시점 학습 로그가 27%였던 것이 이 차이다.

A3.4 CENet 손실 — PPO와 완전히 분리

$$L^{CENet} = 1.0 \cdot \text{MSE}(\text{velocity}, \text{base_lin_vel.detach}())$$
$$+ 1.0 \cdot \text{MSE}(\text{decode}, \text{next_obs.detach}()) \quad \leftarrow \text{비종료 행만(valid_mask)}$$
$$+ \beta \cdot KL, \quad \beta = 1.0$$

$$KL = -0.5 \cdot \text{mean}(\; 1 + \text{logvar} - \text{mean}^2 - \exp(\text{logvar}) \;)$$

별도 Adam(lr 1e-3)으로 **독립적으로** 갱신하고, Actor로 가는 code는 `detach()`되어 PPO 경사가 CENet에 닿지 않는다. 설계 의도는 "정책이 걷기 편한 잠재변수"가 아니라 "관측 히스토리를 잘 설명하는 잠재변수"를 만드는 것이다.

> **valid_mask가 왜 필요한가**
>
> 재구성 타깃은 **다음 스텝의 관측**인데, 이번 스텝에서 에피소드가 끝났다면 그 "다음"은 리셋된 새 에피소드의 첫 관측이다. 이어져 있지 않은 두 상태를 예측하라고 시키는 꼴이라, 종료 행은 재구성 손실에서 제외한다. velocity 손실은 현재 스텝만 쓰므로 이 문제가 없다.

KL은 배치·잠재차원에 대해 **평균**으로 축약한다(합이 아니라). 합으로 두면 latent_dim에 따라 β의 실질 세기가 달라진다.

A3.5 적응형 KL 학습률

학습률은 고정 곡선이 아니라 **KL을 목표 구간에 묶는 피드백 제어기**다.

```
if kl_mean > desired_kl * 2.0:      # 0.02 초과 — 정책이 너무 많이 움직임
    learning_rate = max(1e-5, learning_rate / 1.5)
elif kl_mean < desired_kl / 2.0:    # 0.005 미만 — 너무 안 움직임
    learning_rate = min(1e-2, learning_rate * 1.5)
```

`desired_kl = 0.01`이므로 KL을 0.005~0.02에 묶는다. 조절이 iteration당 한 번이 아니라 **미니배치마다**(5 epoch × 4 minibatch = 최대 20회) 일어나므로, 한 iteration 안에서도 이론상 $1.5^{20} \approx 3{,}300$배까지 움직일 수 있다 — 붕괴가 아주 빠르다.

> **min_learning_rate = 3e-5 가 해결한 것**
>
> rsl-rl에는 `1e-5` 바닥이 하드코딩돼 있다. KL이 계속 크면 LR이 여기까지 밀려 내려와 주차되는데, **주차되면 정책이 안 움직이니 KL이 저절로 작아져 계기판은 정상으로 보인다** — 학습은 멎었는데 경고가 없다. v1 실측에서 한 런이 15,038~26,316 iteration 구간 약 11,000회를 이 상태로 보냈다(전체의 16%).
>
> 현재는 `learning_rate` setter를 가로채 `max(min_lr, value)`를 적용한다 — 업스트림 코드를 고치지 않고 끼워 넣는 방식이다. ARM 1 실측: **1e-5 접촉 0회**, 3e-5 하한 접촉 1,196회. LR을 내리려는 압력 자체는 여전하지만 학습이 멎지 않는다.

A3.6 좌우 대칭 데이터 증강

사족보행은 좌우 대칭이므로, 한 transition을 좌우 반전하면 물리적으로 타당한 또 하나의 transition이 된다. 관측·액션의 각 항목을 대응하는 반대쪽으로 바꾸는 순열을 정의해두고 배치를 2배로 불린다 (`use_data_augmentation=True`). mirror loss(대칭성을 손실로 강제)는 쓰지 않는다 — 증강만으로 충분하고, 손실로 걸면 비대칭이 필요한 상황에서 방해가 된다.

A4. 배포 — 학습 결과를 로봇으로

A4.1 camel.policy.v1 계약

v1에서는 관측 조립·스케일·관절 순서가 배포 코드(C++)에 하드코딩돼 있었다. 지금은 **정책이 자기 규격을 스스로 기술**한다 — ONNX 메타데이터에 JSON 계약이 실리고 로봇이 그것을 읽어 관측을 조립한다. 정책을 추가할 때 C++을 고칠 일이 없다.

계약 항목	내용
관측 term	이름 · 소스 · 크기 · history
transform	term별 `clip` / `scale` / `offset` — 전처리가 코드가 아니라 데이터가 된다
입력 텐서	이름 · 형상 · layout: `latest`(최신 1프레임) / `term_major` / `step_major`
출력 텐서	`actions` 필수, 나머지는 auxiliary
action	type · scale · offset · clip · previous_action · **gains**
joint_names	12개 → 로봇 모터 순서 [HR, HL, FR, FL] × (R, P, K)로 매핑
`policy_dt`	추론 주기. RlWalker 500 Hz 루프의 정수배여야 한다(0.02 → decimation 10)
`states`	순환 상태 입출력 바인딩 — 매 tick 출력을 입력으로 복사(비전 학생 GRU용)

> **전처리 불일치가 구조적으로 불가능해진다**
> 예전에는 학습 쪽(Python)과 배포 쪽(C++)이 각자 같은 전처리를 구현하고 사람이 동기화를 지켰다. 지금은 `clip` → `offset` → `scale`이 계약에 선언되고 로봇이 그대로 적용한다. 학습이 쓴 스케일이 곧 로봇이 쓰는 스케일이다.

A4.2 export 흐름

```
학습 중  train.py → capture_context(env, policy) → runner.deploy_context
                  → 모든 체크포인트의 infos["gd_lab"]["deploy_context"] 에 저장
                    (캡처 실패는 학습을 멈추지 않는다 — 나중에 export 가 거부할 뿐)

export   scripts/export.py
                  → params/agent.yaml 로 네트워크 재구성   ← Kit 불필요, 순수 torch
                  → policy.pt(TorchScript) + policy.onnx + deploy.json
                  → bind_graph() 가 그래프와 계약의 일치를 검증한 뒤 메타데이터 기록

검증     policy-check
                  → 로봇이 실제로 쓰는 PolicyBackend/PolicyRuntime 코드로 계약을 검사
                  → 소스가 하나이므로 "검사 통과"와 "로봇에서 동작"이 같은 질문이 된다
```

> **계약은 체크포인트 안에 있다**
> 학습 시점에 실제 env에서 캡처하므로, 계약이 없는 구버전 체크포인트는 export가 거부한다. `--no-metadata`로 강제하면 계약 없는 맨 그래프가 되어 레거시 경로로만 로드된다.

A4.3 로봇 쪽 백엔드 선택

백엔드	선택 조건	비고
Vendor	디렉터리에 `info.json`	벤더 SDK 규격
Meta	ONNX에 `camel.policy.v1`	권장 경로. `PolicyRuntime`이 계약을 파싱
DreamVrl	계약 없음 + ONNX 입력 **3개**	비전 RL 레거시 경로(현행 GAST 경로는 본편 6부)
Dream	그 외	계약 없는 blind 정책. 입력 폭 45/46으로 payload 유무 자동 감지

계약이 있는데 깨졌다면 폴백하지 않고 **에러**다. 조용히 다른 규격으로 도는 것보다 기동 실패가 낫다는 원칙이다.

A5. 실험 구성

A5.1 TRAIN_ARM — 2×2 요인설계

"arm"은 임상시험 용어의 **실험군**이다. `TRAIN_ARM` 환경변수로 `configs/experiment/arm_N.yaml`을 선택한다.

	50 Hz (decimation 4)	100 Hz (decimation 2)
현재 게인 hip 88.14 / knee 102.22, kd 1.99	ARM 1 (기준군)	ARM 2
nominal 게인 hip 123.39 / knee 127.77, kd 2.4	ARM 3	ARM 4

공통: command pulse OFF, payload ON(0/6 kg), 게인 랜덤화 0.85~1.15배, 지형 10종, cadence swing 0.4 / stance 0.6, 4096 env, seed 42, 40,000 iteration.

> **설계상의 교란 요인**
>
> `num_steps_per_env`가 네 arm 모두 같아서 **100 Hz arm은 같은 iteration 수에서 시뮬레이션 시간이 절반**이다. 100 Hz가 나쁘게 나와도 제어 주기 탓인지 경험량 부족 탓인지 이 실험만으로는 구분되지 않는다.

A5.2 실측 성능 (RTX 5090, 4096 env)

	v1 (48 step)	v2 / ARM 1 (100 step)	배율
collection (물리)	2.088 s	6.637 s	3.18×
learning (PPO)	0.263 s	0.555 s	2.11×
iteration 합계	2.35 s	7.19 s	3.06×
처리량	83,635 step/s	56,955 step/s	0.68×
40,000 iteration	≈26 시간	≈80 시간	

분해하면 **스텝 수 2.083배 × 처리량 저하 1.468배 = 3.06배**로 실측과 정확히 맞는다. 처리량 저하는 커리큘럼 난이도와 무관하다 — 첫 iteration부터 이미 56k이고 terrain_level이 올라가도 변동이 없다(ARM 1: 56,436 @ lv 2.4 → 57,250 @ lv 9.3). 센서 주기 `·dt·decimation·` `render_interval`이 v1과 동일함을 확인했으므로 남은 후보는 지형 10종화에 따른 레이캐스트 비용이지만, **측정으로 확정하지는 않았다**.

> **GPU 선택에 주는 함의**
>
> 시간의 **93%가 물리 시뮬**이고 신경망 학습은 7%다. PhysX는 텐서코어를 쓰지 않고 CUDA 코어·클럭에 민감한 **레이턴시 바운드** 워크로드라, 데이터센터 GPU로 바꿔도 배수가 잘 안 나온다. 이 연구처럼 독립적인 실험군이 여럿일 때는 **카드 성능보다 카드 개수**가 효과적이다 — 느린 카드 4장이 빠른 카드 1장보다 낫다.

A5.3 ARM 1 진행 중 관측 (iteration ≈ 7,600)

지형	평균 레벨	낙상 종료율
pyramid_stairs_inv_wide	10.02	0.170
hf_pyramid_slope_inv	9.61	0.116
pyramid_stairs_inv	9.45	0.119
random_rough	9.41	0.008
hf_pyramid_slope	9.09	0.047
pyramid_stairs_inv_nose	9.09	0.125
pyramid_stairs_wide	9.09	0.050
pyramid_stairs	8.98	0.040
ring_fence	8.94	0.000
pyramid_stairs_nose	8.85	0.085

10종이 **8.85~10.02로 촘촘히 모여 있다** — 뒤처진 지형이 없다. A2.4절에서 설명한 대로 커리큘럼은 모든 env를 능력 한계선에 세워두는 자기균형 장치이므로, 평균 레벨이 9 언저리에서 평평해지는 것은 **병목이 아니라 설계된 평형점**으로 읽는 것이 맞다.

v1 말기에는 같은 지표가 hf_pyramid_slope 43.8%, pyramid_stairs_nose 37.5%로 크게 뒤처져 있었다. 현재는 각각 4.7%, 8.5%다 — LR 하한·롤아웃 길이·cadence·리워드가 동시에 바뀌어서 어느 것의 기여인지는 분리되지 않는다.

2부. 입출력 구조

2.1 Actor 입력 (MLP 235 + 지형 4496)

입력	크기	내용
direct_obs	46	ang_vel 3, projected_gravity 3, command(vx,vy,ωz) 3, joint_pos_rel 12, joint_vel 12, last_action 12, payload 1
cenet_obs	230	위 46 × 최근 5 step(step-major) → CENet → z_t 19 (속도 추정 3 + latent 16, detach)
terrain_latent	32	GAST encoder 출력. 현재 스캔에 valid 셀이 하나도 없으면 정확히 0

actor MLP 입력 = **최근 4프레임 46×4 = 184**(cenet_obs의 최신 4 step, A1.1절과 같은 방식) + z_t 19 + terrain latent 32 = **235** (3906 체크포인트 `actor.0.weight` 512×235로 확인). 배포 스케일(계약): ang_vel ×0.25, command ×(2, 2, 0.25), joint_vel ×0.05, payload ×0.2, 정규화는 그래프 안. 학습 노이즈: ang_vel ±0.2, gravity ±0.05, joint_pos ±0.01, joint_vel ±1.5 (배포는 노이즈 off).

2.2 지형 프레임과 시간 기억

항목	값
격자	몸 yaw 정렬 11(y)×17(x) = 187셀, 10 cm, x −0.8…+0.8 m, y −0.5…+0.5 m, x-fastest
높이 인코딩	`h = 5·clip(scanner_z − z − 0.5, −1, 1)` (기준 = base 원점), 무효 셀 h=0·valid=0. 깊은 갭은 h≈+5
프레임	[h 187 | valid 187 | IR 187] = 561 (+age 1). IR 없는 구버전(oracle)은 374
IR 인코딩	유효·카메라 가시 셀만 0.05 + 0.95·I (I∈[0,1]), 나머지 0
기억	캡처 0.1 s, 8프레임 offset (48, 32, 16, 8, 4, 2, 1, 0) step → 최대 4.8 s 과거. 각 프레임은 캡처 pose와 age를 가지며 **현재 pose로 warp**
학습 텐서	gast_history = 8 × (3·187 + age) = 4496
배포 텐서	frames[1,8,561], poses[1,8,3], pose[1,3], ages[1,8](0–5 s, 빈 슬롯 5) → warp는 encoder ONNX 내부

2.3 출력

- **actions[12]**: 관절 목표 오프셋. 목표 = default + 0.25·a (soft limit 5°). 관절 순서 FL/FR/HL/HR × HIP/THIGH/KNEE, default (0, 0.76, −1.45).
- **PD**: ARM4 — Kp hip·thigh 123.39 / knee 127.77, Kd 2.4, 토크 한계 93.6 / 126 N·m, 100 Hz(sim dt 0.005 × decimation 2), DelayedDCMotor 0–1 step 지연.
- **z_t[19]**: 보조 출력(속도 추정 + 맥락 latent), 진단용.

2.4 Critic (학습 전용, 특권 정보)

고유감각 전부 + base_lin_vel, 발 주변 높이, 발 접촉·접촉력, 마찰계수, 몸통 질량 오프셋, 액추에이터 게인 배율, push Δv, **전체 높이맵(가림·노이즈 없음)**, payload. MLP 512-256-128.

3부. 지형처리기

"지형처리기"는 센서 측정을 2.2절의 11×17 프레임으로 바꾸는 부분이다. 학습 쪽(Isaac, GPU 텐서)과 배포 쪽(매퍼, CPU)이 **같은 프레임 의미**를 만들도록 설계했다.

3.1 학습 쪽: RaycastIrTerrain (`src/gd_lab/gast/rayir.py`)

```
GT 높이 스캔 히트 (11×17)
  → 가시성: 장착 카메라 4대(camera contract vendor_new)에서 각 히트점까지 레이캐스트, 다리 캡슐 가림 → 보이는 셀만
  → 높이 노이즈 N(0, 1 cm) + 에피소드 bias N(0, 1.5 cm)
  → dropout 3%
  → blackout: 스텝당 시작확률 0.001, 길이 50–300 step, 에피소드 전체 5% (IR 포함 전부 0)
  → 무효 셀 h=0 / valid=0 / IR=0
  → 카메라 계약 시계에 맞춰 캡처, 다음 캡처까지 hold (height·mask·IR이 같은 캡처 pose 공유)
```

IR 프록시(셀 단위, gap/지형 라벨 미사용): `I = clip(a + g·ρ(x,y)·max(0, n·l)·(r0/r)^0.5 + N(0,σ), 0, 1)`, a 0.05 · gain 0.57 · r0 0.6 m · σ 0.03 · IR dropout 3%. n = GT 표면 법선, l·r = 그 셀을 보는 가장 가까운 카메라 방향·거리, ρ = 월드 고정 재질장(어두운 패치 약 10%, 밝은 패치 약 15%, 저반사 띠), g = 에피소드 노출. noen rosbag 실측 IR(평균 0.34, p10 0.22, p90 0.46, 포화 없음, 거리 감쇠 (r0/r)^0.5)에 맞췄다.

3.2 배포 쪽: depth+IR → L0S 매퍼 (현행 경로)

현재는 MuJoCo sensor sidecar(`simulation/mujoco/sensor_sidecar/sidecar.py --mode depth_ir`)에서 동작하며, Isaac E1과 **같은 매퍼 코드**(`gd_lab_vrl/tools/rl_replay/`)를 import한다.

단계	모듈	내용
① depth	sidecar / cam130	하부 카메라 4대 160×90, 0.15–5 m. sensor121 잡음: σ 1 cm, flying(모서리) 0.15, spike 0.6%, dropout 3%, 다리 FK 가림
② 이상치 제거	`camfilter136.py`	3×3 이웃(중심 제외) median m, τ = max(0.10 m, 0.05·m), c = τ 안 이웃 수. 유지: (n≥3 이고 (|d−m|≤τ 또는 c≥2)) 또는 (n==2 이고 c==2). 제거 픽셀은 unknown(채우지 않음)
③ 직접 높이	`heightmap119z.py` `h0d`	5 cm 월드 맵, 국소 평면 패치(±0.25 m), DEEP 0.10 m. 정책 셀은 **deepest-of-5**: 중심 + (±2.5, ±2.5) cm 4점 중 가장 낮은 유효 z
④ 구조 갭	`structure130.py`	yaw 정렬 5 cm 레인(±0.55 m), 2 cm bin. 지지 세그먼트를 이어 외삽한 지지면 아래로 레이가 떨어지고 0.25 m 안에 디딤면이 없으면 "빈 공간 증명". no-return은 증거로 쓰지 않음. GAP 셀 → h = 5·clip(scanner_z − (support_z − d_deep) − 0.5), valid=1; pending → 무효. 후방 x < −0.45 또는 10 s 지나면 만료
⑤ 결합	`lidar126.Maps("L0S")`	**DUAL=2 (v2.5-OR)**: 2.5 cm 어긋난 두 격자 중 하나라도 중심이 GAP이면 GAP. **DDEEP=0.7**(d_deep). sidecar가 이 값을 강제(불일치 시 종료)
⑥ IR	sidecar `cell_ir`	3.1과 같은 식(현재 시뮬에서는 코스 GT 기하로 계산). 유효·가시·비드롭 셀만
⑦ 전송	UDP GSC2	"GSC2" | width | stamp_ms | seq | sample x,y,z,yaw | float[561]. 캡처 10 Hz, 전송 지연 0–150 ms, 5% drop, 지도 샘플 100 Hz

LiDAR 경로(`--mode lidar`)는 ①만 다르다: 120×90 빔(0.5°), 피치 50°, 0.1–10 m, 1 cm 잡음, 1% drop, URDF 자기 가림. IR=0.

3.3 매퍼 버전 이력

버전	추가된 것
116	CPU 5 cm 월드 맵(80×80, TTL 2 s), 법선 기반 지지점, IR 어두운 띠(h1) 갭 투표, 센서 오류 모델 `corrupt()`
117 / 118	벡터화 BatchMap / 16 프로세스 병렬 shard, IR 독립 잡음(sensor118)
119z	h0 계열 직접 높이, deepest-of-5(h0d) — 이후 LiDAR에서도 재사용
121	IR-PCA 갭 후보(H3Map), 투표·시간·운동 조건 latch, 출처(UNKNOWN/OBSERVED/INFERRED_GAP), 특권 기하 센서 생성기 sensor121
126 / 127	Isaac/Warp LiDAR(움직이는 URDF 가림) / 2센서
128 / 129	free-space carving(L1/L2), G·GV 진단 arm, L0M
130	**structure130 L0S**, cam130 카메라 경로
136	Ray+IR 교사 지원, **camfilter136**, IR on/zero 스위치
145	매퍼 provenance 기록(opt-in, 5.2절)

3.4 알려진 오차

문제 정의	결과	반영 방향
카메라 spike(0.6%) + min-z 축약기	평지 셀이 "깊음"으로 보여 카메라 갭 성공 20.6%	camfilter136 적용 → 91.8–94.3% (2769), 3906 96.2%
deepest-of-5 샘플링	pit이 5–10 cm 넓게 보임(MuJoCo: 근측 약 5 cm, 착지측 0–10 cm)	aug149에서 이 오차를 증강으로 학습(4.7절). 착지측 오차를 GT로 바꾼 진단 arm E에서 2769 6/6
캡처 지연	LiDAR capture end→통합 p95 126–140 ms, 최신 캡처 age p95 222–242 ms	학습 지연 0–150 ms보다 큼 — depth 캡처를 별도 프로세스로 분리(p95 55–67 ms, late 0)
pose	Pilot이 쓰는 pose와 샘플 pose 차 p95 1.5–2.7 cm, yaw ≤0.2°	보정 불필요

4부. Actor-Critic · PPO 학습

4.1 네트워크 (`GastRayIrActorCritic`)

```
TemporalTerrainEncoder (src/gd_lab/gast/temporal.py), 입력 8 × (3·187 + 1)
  프레임별 CNN : Conv(3→8,3×3) ELU → Conv(8→16,3×3) ELU → AdaptiveAvgPool(5×8) → FC 640→64 ELU
  + age 임베딩 : Linear(2→64) of (age/5, valid 평균)
  → MultiheadAttention(64, 4 heads)  [math SDPA 강제]  → LayerNorm(residual)
  → GRU(64) → FC 64→32 + Tanh  = terrain latent 32   (valid 셀 0개면 0)
보조 decoder : latent 32 → FC 128 → 187×6  (깨끗한 높이·valid·semantic gap·gap-known 재구성)
CENet        : 블라인드와 동일 (enc 128-64, dec 64-128, latent 16, β 1.0, AdaBoot) — 기초편 A1.2–A1.3절
actor        : 235 (46×4 + 19 + 32) → 512 → 256 → 128 → 12 (ELU),  critic: 특권 298 → 512-256-128 → 1
```

terrain latent는 **detach하지 않는다** — PPO 경사가 encoder를 직접 학습시킨다. 보조 재구성 손실(height smooth-L1 + valid/gap BCE, gap 셀 가중 ×5)은 업데이트마다 2048 샘플, 계수 0.1, PPO와 같은 Adam으로 한 번 더 step한다.

4.2 초기화 — 3906 = blind_init

항목	값
출처	DWB-38000 (우리 ARM4 블라인드 DreamWaQ, onvlm2, 4096 env, 0→38000, sha c6cfe178)
복사 / 새로	45개 텐서 복사(actor · CENet · normalizer) / 지형 24개 새로(encoder · decoder)
latent head	actor의 latent 입력 열 0 → 시작 행동 = 블라인드
optimizer	PPO 새로, CENet optimizer 복원 안 함. lr policy 1e-4 / terrain 1e-3, fixed
스크립트	`scripts/gast/train_rayir_gap50_1gpu.sh dwb train` → `train_teacher.py --warm_start_dwb --warm_start_sha256 …`

다른 줄기: 2769 = GAST v2.1 2000에서 warm start(첫 conv 2→3채널, IR 가중치 새로) — 그 계보는 b1_18 ONNX → 17206 → 21068 → GAST 2000. 계보도: `docs/lineage/records_20261016/rbq10_lineage.html`.

4.3 환경 (`Gd-GastRayIrV21Gap50-Rbq10-Dreamwaq-v0`)

항목	값
지형	curriculum 15 rows × 20 cols, 타일 8 m. **platform_gap 10/20 = 50%**(폭 0.02–0.26 m, 단차 0–0.16 m) + 계단 6종(0.05–0.25 m, wide/nose/inv) + rough + 경사 2종 + ring_fence
명령	V21VelocityCommand: heading 명령 100%(stiffness 0.5), standing 20%, vx ±1.2, vy ±1.0, ωz ±1.0, 3–7 s 재샘플. 계단·갭에서 heading을 지형 방향으로 맞춤
도메인 랜덤화	마찰 0.2–1.25, 몸통 질량 ±2 kg · COM · 관성, 게인 ×0.85–1.15, push 10–15 s마다 ±0.5 m/s
페이로드	0 kg 20% / 6 kg 80% (x ±5 cm, z +7 cm), 관측에 포함
핸들 외란	계단 오르막 0–200 N · 0.3–1.0 s · 20–45°(가끔 300–400 N 저크), 내리막 0–150 N · 0.1–0.5 s. 작용점 hip handle (−0.33, 0, 0.12)
종료	time_out(20 s), base_contact, nonfinite_state

4.4 보상 — 비전에서 추가된 항목

기본 locomotion 항목(속도 추종, 자세, 토크·가속·action rate, 발 clearance·slip·cadence 등)은 기초편 A2.3절과 같은 계열이며 가중치만 v2.1 값이다(전체 목록: `GAST_RAYIR_TEACHER_DESIGN_20261018.md` 6.4절).

보상	가중치	의도
platform_gap_foot_drop	−2.0	갭 안으로 발이 떨어짐
platform_gap_intrusion	−3.0	갭 영역 침범
platform_gap_crossing / clean	+0.5 / +0.5	통과 / 침범 없는 깨끗한 통과
gap_foothold_margin · gap_slot_probe	−0.25 · −1.0	모서리 가까운 착지, 갭 더듬기
gap_hind_hop · gap_overlift · gap_foot_clean	−1.0 · −0.25 · +0.25	뒷발 점프, 과도한 발 들기, 깨끗한 발 궤적
stair_foothold_margin · stair_stall	−0.2 · −0.5	계단 모서리 착지, 계단 앞 멈춤
stair_handle_disturbance · push_fall · push_slip	−1.0 · −3.0 · −0.2	핸들 외력 중 자세·넘어짐·미끄러짐

4.5 PPO와 수치 안정성 (`GastPPO`)

horizon 100, 5 epoch × 4 minibatch, lr 1e-3 fixed(하한 3e-5), γ 0.99, λ 0.95, clip 0.2, entropy 0.01, desired_kl 0.01, max_grad_norm 1.0, 대칭 데이터 증강(`mirror_gast`, mirror loss 0). 3906 규모: 1024 env × 5000 iter, 1 GPU(90.111). DDP는 `--distributed --total_envs N`(rank별 divmod 분할).

> **안정성 교훈 (코드에 반영됨)**
>
> - **fused SDPA backward가 inf gradient의 원인**(GAST v2.1 8016/8148 정지) → `GAST_MATH_SDPA=1`로 math kernel 강제. 대칭 증강으로 배치가 2배가 되면 CUDA attention grid 한계를 넘으므로 encoder를 8192 chunk + checkpoint로 실행.
> - **비유한값 가드**: rollout·상태·optimizer에 NaN/inf → 업데이트 skip 후 마지막 정상 스냅샷 복원. 보조 손실은 fp64 norm 검사 후 정상일 때만 rank 평균.
> - **CENet 이상치**: 타깃(velocity/recon/history) clamp로 방어. gradient clipping은 측정상 효과 없음.

4.6 모델 선택 — online Top-5 (`rl/online_top5.py`)

학습 rollout 통계로 후보를 뽑는다(held-out 평가 아님). 조건: iter ≥500, 간격 ≥500, 지형 평균 레벨 ≥9, platform_gap 평균 레벨 ≥8, base contact·gap 종료·계단 종료율 ≤9%, 현 Top-1 대비 gap 성공·contact 회귀 ≤1.0 pp. 점수는 (family, level) 그룹별 가중 macro 평균. 기준 파일: `configs/gast/online_top5_rayir.json`(hot reload). 최종 선택은 E1과 MuJoCo로 한다(5부) — online 점수 1위가 MuJoCo에서 무너진 사례가 있다(A3906 5737: 1/5).

4.7 파인튜: aug149 매퍼 오차 증강 (브랜치 `aug149-mapper-aug`)

목적: Isaac의 깨끗한 Ray 입력과 실제 매퍼(3.2절) 출력 차이를 actor가 견디게 한다. **actor 입력에만** 적용, critic은 clean.

- deepest-of-5 footprint + 에피소드 고정 pit 확장(최종 반경 ≤10 cm), gap 타일에서만 support-0.7 m pit 깊이.
- 군집형 unknown: valid의 5–15%를 blob + 맵 경계 띠로 제거. IR은 유지(무효 셀 0).
- 25% clean 에피소드, 국소 500 업데이트 ramp, 별도 RNG(기존 노이즈 난수열 불변).
- **strict resume**(`--resume_rayir149`): SHA·task 일치 검사, PPO·CENet optimizer와 family 레벨 복원, 조용한 fallback 없음. 주의: `--max_iterations`는 추가분.
- 규모 +2000 iter: A3906(90.111), A2769·C2769 대조(onvlm2).

4.8 카메라 학생 (보류 — 참고)

Ray+IR 이전에는 교사 latent를 카메라 학생으로 증류했다. 최종 학생 GAST-S/B3: 4×80×45 depth+IR → 공유 CNN → 보정된 격자 cross-attention(187셀) → 시간 attention · age 임베딩 · 셀별 GRU(pose warp) → latent 32. 손실 = 고정 actor를 통한 행동 MSE + 0.5 · 기하(가시 셀 높이 · 가시성 BCE) + 0.1 · 품질 BCE, DAgger식 혼합(학생 구동 확률 ramp). 최고 B3 s42 58084: E1 72.6%. 교사 직결(카메라+필터 96.2%)보다 낮아 배포 경로에서 제외했다. 코드: `src/gd_lab/gast/{student,distillation,b3_objective}.py`.

5부. 평가 도구

5.1 E1 — Isaac 페루프 평가 (`experiments/e1/`, `tools/rl_replay/e1_*`)

목적: 정책은 고정한 채 지형 입력 경로만 바꿔(arm), 같은 초기 상태에서 갭·계단 결과를 비교한다. "정책 문제인지 지도 문제인지"를 분리하는 도구다.

항목	값
규모	32 env × 30 s, seed 42–45, vx 1.0 고정(speed ramp 0), dt 0.01 단언, 학생 없음
고정	지형 curriculum off(최고 레벨 고정), 게인 재샘플 동결, 카메라 전송 100 ms 캡처·0–150 ms 지연·5% drop
짝 맞춤	결과마다 initial_state / terrain_grid / packet_schedule SHA 기록. 다른 교사·경로라도 같은 seed면 같은 초기 상태 — 불일치면 요약 거부
지형 family	gap(platform_gap 20열), stdown(pyramid_stairs_inv = pit 탈출), stup(pyramid_stairs)
카메라 프로파일 검사	가중치 옆 `params/env.yaml`의 camera_profile이 env와 같아야 실행(다르면 RuntimeError)

arm	교사에 들어가는 지도
G	GT 깨끗한 높이(valid = GT valid) — 정책 상한
GV	GT를 센서가 2 s TTL 안에 본 셀로 마스크 — 가시성만의 영향
v25or (LOS, DUAL=2, DDEEP=0.7)	배포 매퍼. `U130_SENSOR=lidar|cam`, `U136_CAM_FILTER=1`(camfilter136), `U136_IR=native|zero`
기타	L0/L0D/L0P/L0M/L1/L2(매퍼 변형), h0…h3u(구 카메라 높이맵), `U136_CAM_OFF=spike,…`(잡음 항목별 진단)

지표 (`e1_metrics.py`): 접촉 = 발 힘 >15 N. reached = 앞발이 갭 근측 도달, crossed = base가 원측 +0.8 m·높이 >0.3 m·|pitch|,|roll|<0.5 rad, foot_drop = 발 중심이 갭 안에서 deck 아래, **clean = crossed & foot_drop 없음**. fall(높이 <0.3 m 또는 기울기 >0.5 rad)·stall(2 s에 <5 cm) 이후는 채점 중단, 창 끝까지 못 넘으면 censored. 계단: passed = 외곽 +0.2 m 직립 도달, body_contact = 몸통 힘 >15 N. 지도 진단: 높이 MAE, false-valid, false_gap_5cm, gap recall, 발 레인 유효율.

실행: 단일 `UPDATE116_GPU_LOCK_HELD=1 U130_TEACHER=… U129_FAMILY=gap U130_SENSOR=cam U136_CAM_FILTER=1 bash experiments/e1/run_trial128.sh L0S 42 32 out.json`, 큐 `E1_GPUS="0 1" bash experiments/e1/e1_136.sh jobs.txt 2`(job: name family arm teacher seed ir_mode [sensor] [camoff], 2400 s timeout, result.json 있으면 skip). 출력 `outputs/<name>/result.json`(덮어쓰기 금지 모드). 요약 예: `checkpoints/teachers/gast/rayir_top5_20261016/e1/SUMMARY.md`.

5.2 RL Replay — 보상 해석 뷰어 (`tools/rl_replay/`)

목적: 짧은 rollout(기본 4 s = 400 step) 하나를 녹화해 브라우저에서 로봇 동작과 **보상 분해**를 같이 돌려 본다. "이 상태에서 왜 이 행동을 했나"를 보상·가치로 설명한다. 대량 통계(MuJoCo eval)와 학습 모니터링(TensorBoard)은 대상이 아니다.

구성	내용
녹화	record_rollout.py(Isaac, 교사 구동) / train_student_live.py --replay_out(학생 구동, no_grad). step·env마다 base pose·속도·명령·전 링크 위치·관절·action mean/std·V(s)·latent·보상 항목별 값·접촉·GT 스캔·교사 지형 입력·(선택) 지형 기억·점군. RolloutLog.finish가 G_t, 항목별 G_t^k, TD δ, GAE A_t(γ 0.99, λ 0.95, V(s_T) bootstrap)를 계산
형식	압축 JSON(logs/rl_replay/<name>.json, NaN→null, 덮어쓰기 거부) + live NDJSON 스트림. 8 env·4 s ≈ 16 MB
서버	server.py: stdlib HTTP, 127.0.0.1:8765. /api/recordings, /api/sources, POST /api/record(apptainer로 녹화 실행), /api/live. 작업 1개씩, 대상 GPU에 다른 프로세스가 있으면 거부(fail closed)
뷰어	viewer.html(한국어): three.js + RBQ10 URDF 3D(발 접촉 색), 지형 오버레이(GT / 교사 입력 / 학생 점군), 항목별 보상 막대, G_t 지분, V·G·A·r 타임라인, "지배 보상 항목" 히트맵 + 행동 이벤트(foot_drop, high_lift, hind_hop, fall), 관절별 action ±std·교사 action
매퍼 provenance (145)	sidecar --provenance FILE: 매퍼 결정 이벤트(통합·갭 표시 시도·철회 사유·만료·정책 셀 샘플) JSONL. --raw FILE: 무손실 원시 스트림(R145NPZ1, 프레임별 npz). 기본 off, 켜도 결과 bit 동일(960회 비교)

실행: TRAIN_ARM=4 python3 tools/rl_replay/server.py --port 8767 → http://127.0.0.1:8767/tools/rl_replay/viewer.html. 시뮬레이터 없이 확인: python3 tools/rl_replay/make_demo.py.

5.3 MuJoCo 배포 검증 (gd_rbq10_deploy_vrl/evaluation/)

Isaac과 다른 물리엔진에서 **실제 배포 바이너리**(Pilot C++ + ONNX)와 센서 sidecar로 시험한다. 최종 모델 선택은 여기를 통과해야 한다.

코스 / 도구	내용
gap_stairs_force4	폭 1.8 m, pit 0.65 m. 갭 5/10/15/20/25/30 cm(x 2.0–10.25) → 20 cm 계단 20단 오르기(14.55) → 꼭대기 → 내리기(22.55–28.55) → 25 cm 계단 10단 → 끝 41.55
force4.sh / _repeat / _sensor / _depthir_repeat	4지점 자동 핸들 외력(오르막 당김 15.3, 밂 18.0, 내리막 당김 23.3, 밂 26.0; 당김 뒤 30° 아래, 밂 앞 10° 아래), 1회·반복·센서 경로·고정 seed N회 + PASS/FAIL 영상
gapprefix.sh	갭 구간만(완주 x 12). arm A oracle 전체 잡음 / B 지형 잡음 off / C sidecar 센서 지도 / D 같은 일정의 GT 패킷 / E 착지측 오차만 GT(--map s_landing). SENSOR=lidar|depth_ir, RAW=1
force4_classify.py	궤적 전체에서 첫 사건: FALL(기울기 >70°), GAP-PIT(갭 안 z <0.3 m 1 s 이상), FINISH(x ≥ 40.9), STOP, INVALID. 구역(gapN, up-pull … after)과 통과 갭 수
fall_watch.py	FALL / PIT / STUCK(20 s에 <0.2 m) 즉시 종료 → 반복 시험 가속
진단	pilot_scan_report(패킷 age·pose 차), sidecar_trace_report(캡처 지연 p50/p95), sidecar_map_vs_gt(지도 vs GT: false-gap/false-ground)
리뷰 패키지	deploy/mujoco_review_20261018/<모델>/run_sim.sh [course|--check|stop]: 조이스틱 주행 + 자동 외력. 변수 PAYLOAD_KG(6), FORCE(1), UP_PULL_N/UP_PUSH_N/DOWN_PULL_N/DOWN_PUSH_N(150), FORCE_SEC(2.0), SEED(42)

기록: records/<실험>/runs/<run>/(manifest, state.jsonl 150 Hz, pushes.json, sidecar_*.jsonl/bin, pilot_scan.log, walk.mp4 = MuJoCo 창 + 비전 뷰어 나란히). mp4는 git에 넣지 않는다.

6부. 배포

6.1 export (`gd_rbq10_deploy_vrl/export/gast_teacher_oracle.py`)

```
--checkpoint X.pt --sha256 H --name NAME [--params DIR] --ir_input
  → 학습 코드(GD_LAB_TRAIN_ROOT=../gd_lab_vrl)로 정책 재구성, 체크포인트 SHA·클래스(GastRayIrActorCritic) 검사
  → policy_vrl.onnx/.pt (actor: direct 46, cenet 230, latent 32 → actions 12, z_t 19)
  → gast_encoder.onnx (frames[1,8,561], poses[1,8,3], pose[1,3], ages[1,8] → latent 32; warp 포함)
  → 학습 TerrainHistory + encoder와 무작위 궤적 parity (예: 최대 오차 1.56e-5), blackout 시 latent 0 확인
  → deployment_manifest.json
```

6.2 번들과 계약

번들 `resources/policy/gast/<name>/` = 두 ONNX + `params/{agent,env,gast_warm_start}.yaml` + manifest(교사 SHA, iteration, ONNX SHA, parity, history offsets, capture 0.1 s, camera contract). 실행기: `scripts/gast/teacher/<name>/run_sim_ir.sh`. sim_docker용 계약 `simdocker_contract.json`(schema `simdocker.gast.oracle.v1`, `tools/write_simdocker_contract.py`)에는 관절 순서·default·policy_dt 0.01·action scale 0.25/clip ±5·게인·격자·높이 인코딩·관측 스케일이 들어 있다. camel.policy.v1 GAST runtime은 sim_docker 브랜치 `feat/gast-teacher-runtime`(offline parity ~1e-6, 시뮬 내 smoke 미실시).

6.3 Pilot 런타임 (시뮬레이션)

구성	내용
정책 주기	500 Hz 기준 decimation 5 = 100 Hz (`PolicyBackendVrl.cpp`), 게인은 컴파일 상수(계약과 동일)
encoder 스레드	`VisionStudentThread::gastStep`: 10 ms마다 최신 sim pose(50 ms 넘으면 drop) + 최신 GSC2 패킷(300 ms 넘으면 빈 스캔 → latent 0) → `GastTerrainMemory`(49슬롯 ring, 100 ms push, offset 48…0, 마지막 슬롯 = 현재) → ONNX Runtime CPU 1 thread
latent 신선도	250 ms 이내 정상, 1000 ms 넘으면 fault → Damp. 0 latent로 추론하지 않음
실행 체인	`run_sim_ir.sh` → `scripts/common/launch.sh` → `run_sim_vrl.sh`: Docker MuJoCo(MujocoGastSync, 외력 UDP :19150) + `CAMEL-Pilot --interface lo --sim` + 비전 뷰어 + CAMEL-Console

6.4 현재 한계

- **GAST 경로는 시뮬레이션 전용이다.** Pilot은 `--interface lo --sim`이 아니면 GAST 모델을 거부한다. 학습·E1·MuJoCo·sim_docker는 모두 정답(GT) pose를 쓴다(2026-10-16 결정). 실기 pose는 camel_pilot이 발행하는 `/odom`(nav_msgs/Odometry, odom→base_link, 50 Hz; rosbag에서 확인)을 쓸 예정이며, 모델의 pose 입력 형식(캡처마다 world x, y, yaw)은 이미 맞춰 두었다. **실기 포팅 때 할 일**: Pilot에 `/odom` 구독 추가, 실기 odom 오차 측정 후 pose 오차 평가(UPDATE157, `POSE_ERR`)로 민감도 확인, 실제 D4xx 스트림 → 매퍼 연결.
- sidecar의 IR은 코스 GT 기하로 계산한다(실제 IR 영상 아님). depth 잡음에 stereo fill-in 패치는 없다.
- 계약은 history 초기값을 `zeros`로, launcher는 `RBQ_VRL_HISTORY_INIT=repeat_first`를 쓴다 — 하나로 맞춰야 한다.
- force4_classify의 갭 창과 코스 XML 표식이 약간 다르다 — 정리 필요.

7부. 현재 결과와 과제

문제 정의	결과	반영 방향
3906 Isaac 성능 (E1, seed 42–45)	v25or LiDAR 갭 90/93 = 96.8%, 카메라+필터 76/79 = 96.2%, 계단 100%	현재 기준선
매퍼 오차 증강 효과 (aug149)	A3906 E1 카메라+필터 갭 98.7–100%. A2769 ≈ C2769(대조)	Isaac 개선이 MuJoCo로 그대로 이어지지 않음 → MuJoCo 필수
MuJoCo depth_ir 갭 (12 seed)	A2769_top1 11/12, 원래 3906 10/12, A3906 l2000 10/12, A3906 5737 1/5	최종 배포 후보: A2769_top1 vs 원래 3906 (결정 대기)
핸들 외력 (force4, 200 N × 2 s)	학습 범위(≤1 s) 초과 조건. 3906은 오르막 당김에 약함(센서 10/20)	150 N × 2 s로 재측정 예정(100회 반복은 중단 상태)
실기 배포	시뮬레이션 전용 경로	실기 pose·센서 연결, sim_docker runtime smoke 후 PR
다음 기능	좁은 통로·시계방향 회전·계단 정렬 NavSkill 설계 v0.2 검토 중	GAST_NAVSKILL_DESIGN_20261018.md

부록. 코드 위치

영역	경로
교사 관측·모델·PPO	gd_lab_vrl/src/gd_lab/gast/{rayir,temporal,teacher,observations,geometry,warm_start,tasks}.py
CENet·runner·Top-5·DDP	gd_lab_vrl/src/gd_lab/rl/{cenet,actor_critic,ppo,runner,online_top5,distributed}.py
갭·계단 MDP	gd_lab_vrl/src/gd_lab/mdp/{gap_stair_v21,platform_gap*}.py
카메라 계약	gd_lab_vrl/src/gd_lab/core/camera_{contract,geometry,timing}.py (vendor_new 기본, vendor_legacy는 학습 금지)
학습 실행기	gd_lab_vrl/scripts/gast/{train_teacher.py, train_rayir_gap50_1gpu.sh, train_rayir_from2000_1gpu.sh}
지형처리기·E1·Replay	gd_lab_vrl/tools/rl_replay/, gd_lab_vrl/experiments/e1/
가중치·E1 결과	gd_lab_vrl/checkpoints/teachers/gast/rayir_top5_20261016/
설계·계보	gd_lab_vrl/docs/design/GAST_RAYIR_TEACHER_DESIGN_20261018.md, docs/lineage/records_20261016/
export·번들·Pilot	gd_rbq10_deploy_vrl/{export, resources/policy/gast, perception/common/VisionStudentThread.cpp, perception/oracle/GastTerrainMemory.hpp}
sidecar·MuJoCo 평가	gd_rbq10_deploy_vrl/simulation/mujoco/sensor_sidecar/sidecar.py, gd_rbq10_deploy_vrl/evaluation/

근거: 위 경로의 소스와 3906 params/{env,agent,gast_warm_start}.yaml, E1 SUMMARY.md, docs/experiments/RAYIR_MUJOCO_SENSOR_20261016.md, MuJoCo 기록. 학습 장비: 로컬 RTX 5090, 90.111(RTX 5090), onvlm2(다중 GPU).